

1. What is a Programming Language?

A **Programming Language** is a language used by humans to communicate with computers.

Just like humans use English, Hindi, or Marathi to communicate with each other, programmers use programming languages to give instructions to computers.

A computer only understands **Machine Language (0 and 1)**, so programming languages help us write instructions in a human-readable format.

Example

```
print("Hello World")
```

Here, we are instructing the computer to display "Hello World" on the screen.

Why Do We Need Programming Languages?

- To communicate with computers
- To build software and applications
- To automate tasks
- To process and analyze data
- To create websites and mobile applications

Popular Programming Languages

- Python
 - Java
 - C
 - C++
 - JavaScript
 - C#
 - PHP
-

2. Python History

Who Created Python?

Python was created by Guido van Rossum in the late 1980s.

He was working at a research institute in the Netherlands and wanted a programming language that was:

- Easy to learn
- Easy to read
- Powerful

- Flexible
-

Why Was Python Created?

At that time, languages like C and C++ were powerful but difficult for beginners.

Guido wanted a language that:

- Had simple syntax
 - Reduced coding complexity
 - Improved productivity
 - Allowed programmers to focus on solving problems
-

Origin of the Name "Python"

Many people think Python is named after the snake. Actually, it was inspired by a British comedy show called Monty Python's Flying Circus .Guido liked the show and chose the name "Python".

Important Python Versions

Python 1.0 (1994)

Features:

- Functions
 - Modules
 - Lists
 - Dictionaries
-

Python 2.0 (2000)

Features:

- List Comprehension
 - Garbage Collection
 - Better Memory Management
-

Python 3.0 (2008)

Features:

- Cleaner Syntax

- Better Unicode Support
- Improved Performance
- Future-Oriented Version

Python 2 is now discontinued.

Today, developers use Python 3.x versions.

3. What is Python?

Python is a **High-Level, General-Purpose, Interpreted Programming Language**.

It is one of the most popular programming languages in the world.

Definition

Python is used to develop:

- Web Applications
 - Desktop Applications
 - Data Analysis
 - Machine Learning
 - Artificial Intelligence
 - Automation Scripts
 - Scientific Applications
-

Features of Python

1. Easy to Learn

Python syntax is simple and readable.

Example:

```
print("Hello")
```

2. Easy to Read

Python code looks similar to English.

Example:

```
age = 25
if age >= 18:
    print("Adult")
```

3. Open Source

Python is free to use.

4. Platform Independent

A Python program can run on:

- Windows
- Linux
- macOS

without changing the code.

5. Interpreted Language

Python executes code line by line.

No separate compilation is required.

6. Object-Oriented

Python supports classes and objects.

7. Large Community Support

Millions of developers use Python worldwide.

8. Huge Library Support

Examples:

- NumPy
 - Pandas
 - Matplotlib
 - TensorFlow
 - Scikit-Learn
-

4. Compiler vs Interpreter

Both are used to convert source code into machine code.

Compiler

A compiler translates the entire program at once before execution.

Working

Source Code

↓

Compiler

↓

Machine Code

↓

Execution

Examples

- C
- C++
- Go

Advantages

- Faster execution
- Errors displayed after compilation

Disadvantages

- Compilation takes time
- Entire program must be compiled

Interpreter

An interpreter translates and executes code line by line.

Working

Source Code

↓

Interpreter

↓

Line-by-Line Execution

Examples

- Python
- JavaScript
- Ruby

Advantages

- Easy debugging
- Immediate execution

Disadvantages

- Slower than compiled languages

Compiler vs Interpreter Table

Feature	Compiler	Interpreter
Translation	Entire Program	One Line at a Time
Speed	Faster	Slower
Error Detection	After Compilation	Line by Line
Output	Machine Code File	No Separate File
Examples	C, C++	Python, JavaScript

5. How Python is Platform Independent?

What is Platform Independence?

A program that runs on multiple operating systems without changing the code is called **Platform Independent**.

Example:

A Python program written on Windows can also run on Linux and macOS.

How Does Python Achieve Platform Independence?

Step 1: Write Python Code

```
print("Hello Python")
```

This is called **Source Code**.

Step 2: Python Compiler Creates Bytecode

Python converts source code into:

Bytecode

A file with extension:

.pyc

is generated internally.

Step 3: Python Virtual Machine (PVM)

The Bytecode is executed by the **Python Virtual Machine (PVM)**.

Source Code (.py)

↓

Python Compiler

↓

Bytecode (.pyc)

↓

PVM

↓

Machine Code

↓

Output

Why Platform Independent?

Every operating system has its own Python Virtual Machine.

Windows

Bytecode

↓

Windows PVM

↓

Output

Linux

Bytecode

↓

Linux PVM

↓

Output

macOS

Bytecode

↓

macOS PVM

↓

Output

Since the same bytecode can run on different PVMs, Python becomes platform independent.

